Database Design & Normalization Assignment

1. Introduction

This project models two real-world systems:

1. Smart Campus Hostel Management
2. Mini E-Commerce Platform

2. Smart Campus Hostel Management

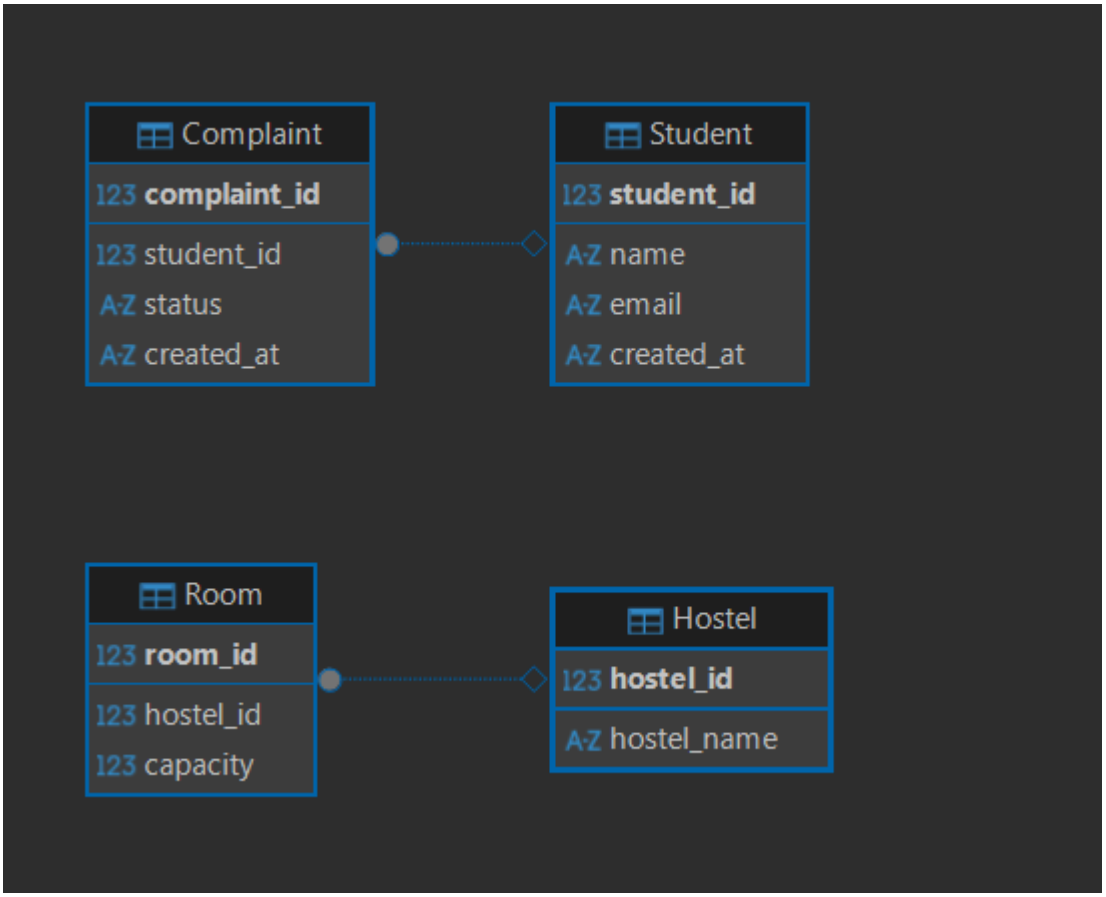
Entities & Attributes

- Student (student_id, name, email)
- Hostel (hostel_id, hostel_name)
- Room (room_id, hostel_id, capacity)
- Complaint (complaint_id, student_id, status)

Relationships

- Hostel → Room (1)
- Room → Student (1)
- Student → Complaint (1)

ER Diagram




```sql
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    created_at TIMESTAMP
);

CREATE TABLE Hostel (
    hostel_id INT PRIMARY KEY,
    hostel_name VARCHAR(100)
);

CREATE TABLE Room (
    room_id INT PRIMARY KEY,
    hostel_id INT,
    capacity INT,
    FOREIGN KEY (hostel_id) REFERENCES Hostel(hostel_id)
);

CREATE TABLE Complaint (
    complaint_id INT PRIMARY KEY,
```

```
    student_id INT,
    status VARCHAR(50),
    created_at TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES Student(student_id)
);

INSERT INTO Hostel VALUES (1,'Block A');
INSERT INTO Room VALUES (1,1,3);
INSERT INTO Student VALUES (1,'A','a@gmail.com',CURRENT_TIMESTAMP);
INSERT INTO Complaint VALUES (1,1,'Pending',CURRENT_TIMESTAMP);

SELECT * FROM Complaint;

INSERT INTO Student VALUES
(1,'Aman','aman@gmail.com',CURRENT_TIMESTAMP),
(2,'Riya','riya@gmail.com',CURRENT_TIMESTAMP),
(3,'Kiran','kiran@gmail.com',CURRENT_TIMESTAMP),
(4,'Neha','neha@gmail.com',CURRENT_TIMESTAMP),
(5,'Rahul','rahul@gmail.com',CURRENT_TIMESTAMP),
(6,'Sneha','sneha@gmail.com',CURRENT_TIMESTAMP),
(7,'Arjun','arjun@gmail.com',CURRENT_TIMESTAMP),
(8,'Pooja','pooja@gmail.com',CURRENT_TIMESTAMP),
(9,'Vikram','vikram@gmail.com',CURRENT_TIMESTAMP),
(10,'Anjali','anjali@gmail.com',CURRENT_TIMESTAMP);

INSERT INTO Hostel VALUES (1,'Block A');
INSERT INTO Hostel VALUES (2,'Block B');

INSERT INTO Room VALUES
(1,1,3),
(2,1,2),
(3,2,3),
(4,2,2);

INSERT INTO Complaint VALUES
(1,1,'Pending',CURRENT_TIMESTAMP),
(2,2,'Resolved',CURRENT_TIMESTAMP),
(3,3,'Pending',CURRENT_TIMESTAMP),
(4,4,'In Progress',CURRENT_TIMESTAMP),
(5,5,'Resolved',CURRENT_TIMESTAMP),
(6,6,'Pending',CURRENT_TIMESTAMP),
(7,7,'Resolved',CURRENT_TIMESTAMP),
(8,8,'In Progress',CURRENT_TIMESTAMP),
```

```sql
(9,9,'Pending',CURRENT_TIMESTAMP),
(10,10,'Resolved',CURRENT_TIMESTAMP);

-- 1. View all students
SELECT * FROM Student;

-- 2. Count total complaints
SELECT COUNT(*) FROM Complaint;

-- 3. Pending complaints
SELECT * FROM Complaint WHERE status='Pending';

-- 4. Complaints per student
SELECT student_id, COUNT(*)
FROM Complaint
GROUP BY student_id;

-- 5. Students with complaints
SELECT s.name, c.status
FROM Student s
JOIN Complaint c ON s.student_id = c.student_id;

-- 6. Rooms per hostel
SELECT hostel_id, COUNT(*)
FROM Room
GROUP BY hostel_id;

-- 7. Students with email filter
SELECT * FROM Student
WHERE email LIKE '%gmail.com';

-- 8. Complaint status count
SELECT status, COUNT(*)
FROM Complaint
GROUP BY status;
```

status	COUNT(*)
In Progress	2
Pending	4
Resolved	4

complaint_id	student_id	status	created_at
1	1	Pending	2026-04-27 00:35:01
3	3	Pending	2026-04-27 00:35:01
6	6	Pending	2026-04-27 00:35:01
9	9	Pending	2026-04-27 00:35:01

student_id	COUNT(*)
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1

name	status
Aman	Pending
Riya	Resolved
Kiran	Pending
Neha	In Progress
Rahul	Resolved
Sneha	Pending
Arjun	Resolved
Pooja	In Progress
Vikram	Pending
Anjali	Resolved

hostel_id	COUNT(*)
1	2
2	2

3. Mini E-Commerce Platform

Entities & Attributes

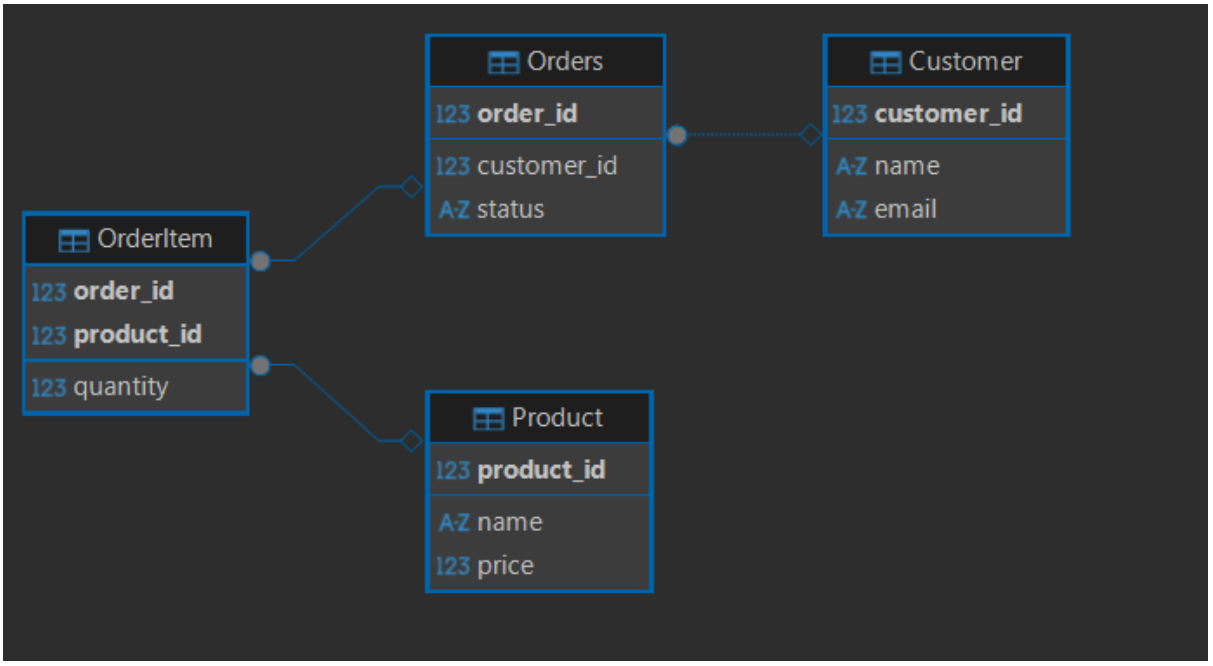
- Customer (customer_id, name, email)
- Product (product_id, name, price)

- Orders (order_id, customer_id, status)
- OrderItem (order_id, product_id, quantity)

Relationships

- Customer → Orders (1)
- Orders ↔ Products (M via OrderItem)

ER Diagram




```sql
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);

CREATE TABLE Product (
    product_id INT PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10,2)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    status VARCHAR(50),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
```

```
);

CREATE TABLE OrderItem (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY(order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);

INSERT INTO Customer VALUES (1,'John','john@gmail.com');
INSERT INTO Product VALUES (1,'Phone',10000);
INSERT INTO Orders VALUES (1,1,'Placed');
INSERT INTO OrderItem VALUES (1,1,2);



INSERT INTO Customer VALUES
(2,'Aman','aman@gmail.com'),
(3,'Riya','riya@gmail.com'),
(4,'Kiran','kiran@gmail.com'),
(5,'Neha','neha@gmail.com'),
(6,'Rahul','rahul@gmail.com'),
(7,'Sneha','sneha@gmail.com'),
(8,'Arjun','arjun@gmail.com'),
(9,'Pooja','pooja@gmail.com'),
(10,'Vikram','vikram@gmail.com');

INSERT INTO Product VALUES
(1,'Phone',10000),
(2,'Laptop',50000),
(3,'Headphones',2000),
(4,'Keyboard',1500),
(5,'Mouse',800),
(6,'Monitor',12000),
(7,'Tablet',15000),
(8,'Charger',500),
(9,'Camera',25000),
(10,'Speaker',3000);
INSERT INTO Orders VALUES
(1,1,'Placed'),
(2,2,'Shipped'),
```

```
(3,3,'Delivered'),
(4,4,'Placed'),
(5,5,'Delivered'),
(6,6,'Shipped'),
(7,7,'Placed'),
(8,8,'Delivered'),
(9,9,'Placed'),
(10,10,'Shipped');
INSERT INTO OrderItem VALUES
(1,1,2),
(1,3,1),
(2,2,1),
(3,4,2),
(4,5,3),
(5,6,1),
(6,7,2),
(7,8,5),
(8,9,1),
(9,10,2);
-- 1. View all customers
SELECT * FROM Customer;

-- 2. Orders with customer names
SELECT o.order_id, c.name
FROM Orders o
JOIN Customer c ON o.customer_id = c.customer_id;

-- 3. Total orders
SELECT COUNT(*) FROM Orders;

-- 4. Orders by status
SELECT status, COUNT(*)
FROM Orders
GROUP BY status;

-- 5. Products in each order
SELECT o.order_id, p.name, oi.quantity
FROM OrderItem oi
JOIN Product p ON oi.product_id = p.product_id
JOIN Orders o ON oi.order_id = o.order_id;

-- 6. Total quantity sold per product
SELECT product_id, SUM(quantity)
```

```
FROM OrderItem
GROUP BY product_id;

-- 7. Delivered orders
SELECT * FROM Orders
WHERE status='Delivered';

-- 8. High value products (>10000)
SELECT * FROM Product
WHERE price > 10000;
```

order_id	name
1	John
2	Aman
3	Riya
4	Kiran
5	Neha
6	Rahul
7	Sneha
8	Arjun
9	Pooja
10	Vikram

COUNT(*)
10

status	COUNT(*)
Delivered	3
Placed	4
Shipped	3

order_id	name	quantity
1	Phone	2
1	Headphones	1
2	Laptop	1
3	Keyboard	2
4	Mouse	3
5	Monitor	1
6	Tablet	2
7	Charger	5
8	Camera	1
9	Speaker	2

order_id	customer_id	status
3	3	Delivered
5	5	Delivered
8	8	Delivered

product_id	name	price
2	Laptop	50,000
6	Monitor	12,000
7	Tablet	15,000
9	Camera	25,000